GNNs for Rising Stars 2026 — Learner Answer Sheet

Name: Faran Taimoor Butt **Email:** faranbutt789@gmail.com
Country of residence: Pakistan **Date:** 31Dec2025

Submission deadline: Jan 7, 2026 at 8pm (UK)

Submission Form: https://docs.google.com/forms/d/1qsLPMqy1D3ByJVleTbwSgASUleHDuh23R4mbp6NaORI

Section 1: Paper Summarization

Your first task is to **read and summarize 5 assigned papers** according to the guidelines provided below. The URL links to the papers are provided below.

Instructions

- **Review the guidelines carefully** to ensure all required elements are included in each section of your summary.
- **Replace these instruction lines** and begin typing your summary content directly into the document.
- **Content Length:** Each paper summary should aim for approximately **one full page** (excluding any visual aids).
- **Visuals:** Feel free to **incorporate figures, your own handwritten summaries, or mind maps** to enhance your understanding and presentation.

1. Skim First Before diving in deeply:

- **Abstract:** Get the gist of the problem, approach, and results.
- **Figures and Tables:** Quickly see main experiments and findings.
- **Conclusion:** Reinforces the key achievements of the paper.

2. Read Strategically

- **Introduction:** Understand the motivation, the problem, and why it matters.
- **Related Work:** Skim to see how this paper differs from previous work.
- **Methods:** Focus on what the model or approach does, not every technical detail.
- **Experiments** / **Results:** Note key metrics, datasets, and comparisons.
- *Tip:* Don't get stuck on formulas. Highlight unfamiliar terms and return later.

Summary (Objectihttps://www.youtube.com/watch?v=zkFerNEVvtYlist=PL0vKVrkG4hWrLHcaP4xsDl-8tXX1lkbgFve)

- Goal / Problem: What question does the paper answer?
- Method / Approach: How do the authors solve it?
- Main Results: What evidence supports their claims?
- Contributions: What's new compared to previous work?

Reflection (Subjective)

- Interesting Insights: What surprised or inspired you?
- Difficult Points: What was unclear or hard to understand?
- Connections: How does this relate to your prior knowledge or other ideas you have?
- Next Steps / Extensions: How would you build on this work?

4. Write Concisely

- Use your own words; avoid copy-pasting sentences.
- Focus on ideas and reasoning, not exact wording.

5. Engage Critically

- Ask: Does the method make sense? Are the experiments convincing?
- Compare with other papers in the field.
- Think: "If I had to explain this in 5 minutes, what would I say?"

6. Review and Refine

- Re-read your summary after a day to check clarity, grammar and punctuation.
- Highlight gaps or questions to revisit.

Quick Tips

- Paper hierarchy: Abstract → Figures → Conclusion → Full text.
- Highlight strategically: Only key sentences.
- One paper may take multiple passes; that's normal.

Paper 1: Title

Paper URL: https://openreview.net/forum?id=ijK5hyxs0n
Paper Title: Graph Metanetworks for Processing Diverse Neural Architectures
Authors: Derek Lim, Haggai Maron, Marc T. Law, Jonathan Lorraine, James Lucas
Conference: ICLR 2024

Goal / Problem:
The paper discusses the problem of designing metanetworks neural networks that operate directly on the parameters of other neural networks.Its like training neural network based on the data that is the other neural network in question.Prior metanetworks like (Neural Functional Network(NFN), DWSNets are limited to simple architectures like MLP or basic CNNs without normalization or attention layers.They struggle with more complex real world architecture due to difficulty in handling parameter symmetries and architectural diversity.This paper asks a very critical question "Can we built a single general metanetwork framework that respects symmetries and work across diverse modern architectures like Transformers?"

Proposed Method:
The authors introduced Graph Meta Networks(GMNs).The key idea presented in the paper is to represent any feed forward neural network as a parameter graph.After that apply the Graph

Neural Network (GNN) to process it.

Parameter Graph: Unlike standard computation graph that explode in size due to weight sharing (i.e convolutions) GMNs use a compact parameter graph where each parameter corresponds to exactly one edge. The representation for

- **Linear Layers:** Weights as edges between neuron nodes, biases as edges from dedicated bias nodes.
- **Convolution Layers:** multigraph with nodes for input/output channels; kernel positions encoded in edge features.
- **Attention:** nodes for feature dimensions; edges for query/key/value and output projections
- **Residual connections:** parameter free edges with fixed weight
- **Normalization Layers:** separate nodes for learnable scale/shift parameters.

Neural DAG Automorphisms: The research also focuses on formalizing symmetries as graph automorphisms that preserve node types and weight sharing constraints.They prove GMNs are equivariant to these symmetries i.e permuting input network parameters induces a consistent permutation in GMN outputs. **GNN Processing:** A message passing GNN updates nodes, edge and optionally global features. Edge features hold parameter values.Predictions are made by pooling edge representations/

Main Results:

Accuracy Prediction (CIFAR-10) from Figure 5 in paper shows Graph MetaNetworks outperform baseline (DeepSets, DMC) on:

- **Varying CNN** : Networks with different depths, widths and normalization types
- **Diverse Architectures:** Includes Resnets, Vision Transformers, 1D/2D CNNs and DeepSets.
- Performance gains are especially strong in low data and out-of-distribution(OOD) settings (e.g training on narrow networks, testing on wide ones)

	Metanet	Varying CNNs R^2	Varying CNNs τ	Diverse Architectures R^2	Diverse Architectures τ
50%	DeepSets (Zaheer et al., 2017)	.778±.002	.697±.002	.562±.020	.559±.011
	DMC (Eilertsen et al., 2020)	.948±.009	.876±.003	.957±.009	.883±.007
	GMN (Ours)	**.978**±.002	**.915**±.006	**.975**±.002	**.908**±.004
5%	DeepSets (Zaheer et al., 2017)	.692±.006	.648±.002	.126±.015	.290±.010
	DMC (Eilertsen et al., 2020)	.816±.038	.762±.014	.810±.046	.758±.013
	GMN (Ours)	**.876**±.010	**.797**±.005	**.918**±.002	**.828**±.005
OOD	DeepSets (Zaheer et al., 2017)	.741±.015	.683±.005	.128±.071	.380±.014
	DMC (Eilertsen et al., 2020)	.387±.229	.760±.024	−.134 ± .147	.566±.055
	GMN (Ours)	**.891**±.037	**.870**±.010	**.768**±.063	**.780**±.030

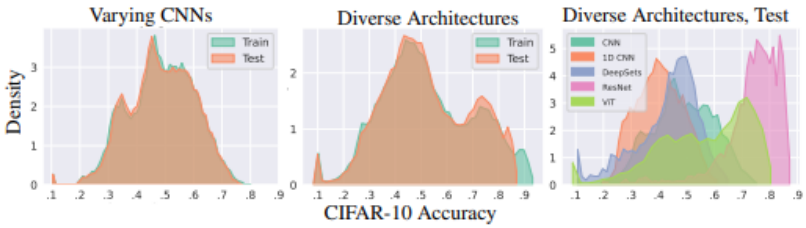


Figure 5: Histograms of CIFAR-10 accuracies for our Varying CNNs and Diverse Architectures datasets. Left and middle show train and test accuracy for the two datasets. Right shows test accuracy of Diverse Architectures split by model type.

2D INR Editing: On transforming implicit neural representations (e.g contrast/dialation) GMNs match or beat specialized methods like NFN and NFT.

4.2 Editing 2D INRs

Table 2: Test MSE (lower is better) for editing 2D INRs, following the methodology of (Zhou et al., 2023a). Results of baselines are from (Zhou et al., 2023a;b).

Metanetwork	Contrast	Dilate
MLP	.031	.306
MLP-Aug	.029	.307
NFN-PT	.029	.197
NFN-HNP	.0204 ±.0000	.0706 ±.0005
NFN-NP	.0203 ±.0000	.0693 ±.0009
NFT	.0200 ±.0002	**.0510** ±.0004
GMN (ours)	**.0197** ±.0000	.0603 ±.0010

Table 3: Results for self-supervised learning of neural net representations, in test MSE of a linear regressor on the learned representations. Numbers besides GMN from Navon et al. (2023).

Metanetwork	Test MSE
MLP	7.39 ±.19
MLP + Perm. aug	5.65 ±.01
MLP + Alignment	4.47 ±.15
INR2Vec (Arch.)	3.86 ±.32
Transformer	5.11 ±.12
DWSNets	1.39 ±.06
GMN (ours)	**1.13** ±.08

Self Supervised Learning: Graph Meta Networks learn better represent of Multi Layer Perceptron trained on sinusoids achieving lower test MSE.This conclusion is based on the comparison with less flexible permutation equivariant metanet DWS-Nets in a self supervised learning experiments. The data consists of MLPs trained to represent sinusoidal functions of the form $x \to asin(bx)$ where a and b vary.

The goal was to learn network representations using contrastive learning approach similar to SimCLR.Graph Meta Networks learn representations for edges in a network and then average to get a single vector for each network.

For evaluation a simple linear model was trained to predict the sinusoidal parameters a and b. As shown in Table 3, GMNs outperform all baselines, showing strong performance even in settings designed to favor competing methods.

Expressive Power of GMNs: GMNs are equivariant to parameter symmetries and at least as expressive as prior meta networks (StanNN, NP-NFN) on simple architectures.

Contributions:

The key contributions this research makes are :

- First metanetwork that handles normalization layers, attention, residual blocks and group equivariant layers in a unified way.
- Efficient graph construction that avoids scaling issues from weight sharing
- Formalizes symmetries via Neural DAG Automorphisms and proves equivariant and expressivity.
- Strong results across diverse tasks and architectures where prior methods fail.

Reflection 1

Interesting ideas:

- The insight that any neural network can be viewed as a graph and that GNNs naturally respect its symmetries is indeed elegant and powerful
- Using bias nodes instead of self loops to enable communication between bias paramters is a subtle but important design choice for expressivity.

Difficult concepts:

- The distinction between computation graphs and parameter graphs is crucial for this paper.It was initially confusing but especially for convolution/attention layers.
- Neural DAG Automorphisms concept is difficult to grasp as it provides a mathematical framework for identifying symmetries across diverse neural network architectures.While previous research focus on simple neuron swapping MLPs this paper addresses the complexity of modern networks like residual connections, weight sharing in CNNs and multi head attention.The authors introduced concept of Parameter Graphs a representation that maps parameters to edges rather than activations.They proved that by treating these architectures as graphs, Graph Neural Networks can remain equivariant to these symmetries.

Connections:
The work sits at the intersection of meta learning, geometric deep learning and neural architecture analysis. It connects with the ideas of neural network pruning (removal of least important neural network parameters) and program synthesis(code structure is represented as graphs).
Personal Takeaways:

- Graph Meta Networks can be utilized for model diagnosis.
- As GMN offer a plug and play way to analyze arbitrary networks which are useful for meta learning and pruning
- If any model can be turned into a graph, then GNNs become Swiss Army Knife for analyzing,editing or generating models regardless of internal complexity.

Critical Thoughts:
While GMNs is impressive in handling complex model archetechures the current method dosent handle non-parameteric operations like max-pooling and padding.
Also the claim on OOD is strong but would GMNs really extrapolate to, say a 100 layer Transformer if trained only on 2 to 4 layers? Things like (GNN depth or impact of edge features) would strength this claim.
Future Work: The work can be extended to non-feed forward neural networks such as RNN,LSTM or GRU. We can also scale to billion parameter models using sparse or hierarchical GNN. We can also explore Graph Transformers for even better expressivity.

Paper 2: Title

Paper URL: https://openreview.net/forum?id=4IT2pgc9v6

Paper Title: One For All.Towards Training One Graph Model for All Classification Tasks

Authors: Hao Liu[1], Jiarui Feng[1], Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, Muhan Zhang

Conference: ICLR 2024

Goal / Problem:
This papers addresses one of the fundamental challenge in graph machine learning that "Can we make a unified model that can process diverse graph data across different domains and handle multiple task types (node classification, link prediction, graph classification). Unlike language data which has uniform representation on the other hand graph data from different domains (Molecular graph, Citation graphs etc) have vastly different feature representations making cross domain learning difficult. Additionally, different graph tasks require different methodologies and there is no established paradigm for in context learning in graphs comparable to prompt engineering in NLP domain.

Proposed Method:
The authors introduced One for All (OFA) a framework with three key innovations:

- **Text Attributed Graphs(TAG):** All graphs regardless of domain are converted into a uniform format where every node and edge are described in natural language.OFA rewrites every node and edge as human readable sentence.For example For example:

  Carbon Atom might become:

  - "Feature node: Carbon, aromatic, in a six memeber ring etc.

  Citation edge becomes:

  - Feature edge: Paper A cites Paper B on attention mechanism.

  Then a single LLM like (BERT or LLama 2) converts all these descriptions into vectors in a shared embedding space.Suddenly a molecule and research paper live in the same representation space making joint training possible.

- **Nodes of Interest:** The propsed concept is you should focus what matters.

  Graph tasks differ in scope:

  - **Node tasks** care about target node itself.
  - **Link tasks** care about pair of nodes.
  - **Graph tasks** care about all nodes in the graph

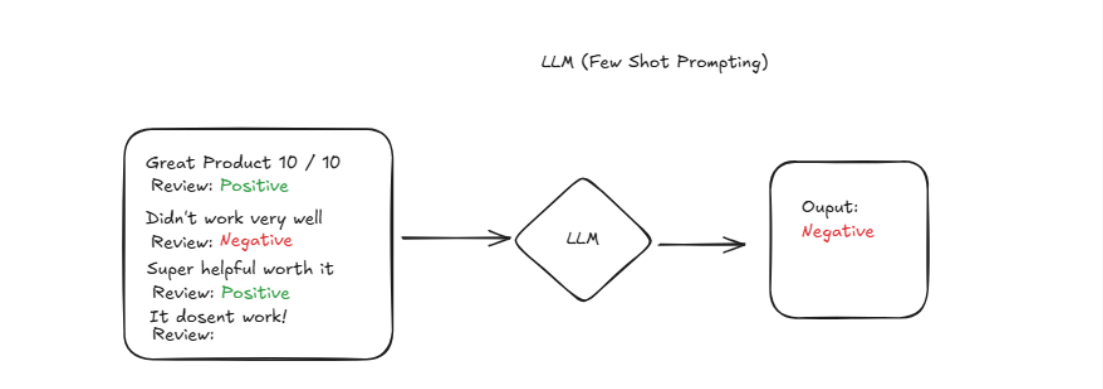
  OFA unifies them by always extracting a subgraph around the **"Nodes of Interest"**.

  For example

  - For a paper category paper prediction (NOI = "that paper")
  - Do these two papers cite each other? (NOI = Paper A, Paper B)
  - "Is this molecule toxic ? " (NOI = all atoms)

  This give every task the same input structure, so one model can handle them all.

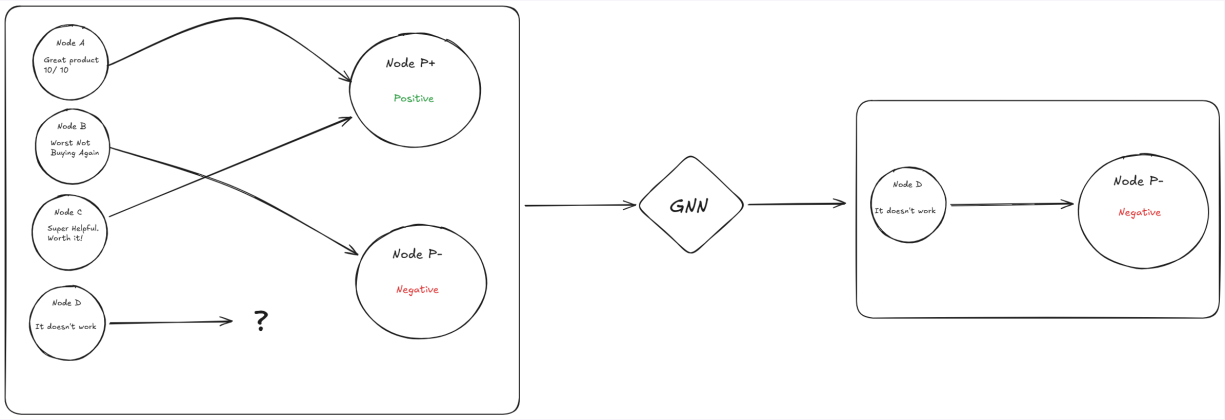
Graph Prompting Paradigm (GPP): This part is inspired by LLM prompting techniques like zero shot prompting or few short prompting.



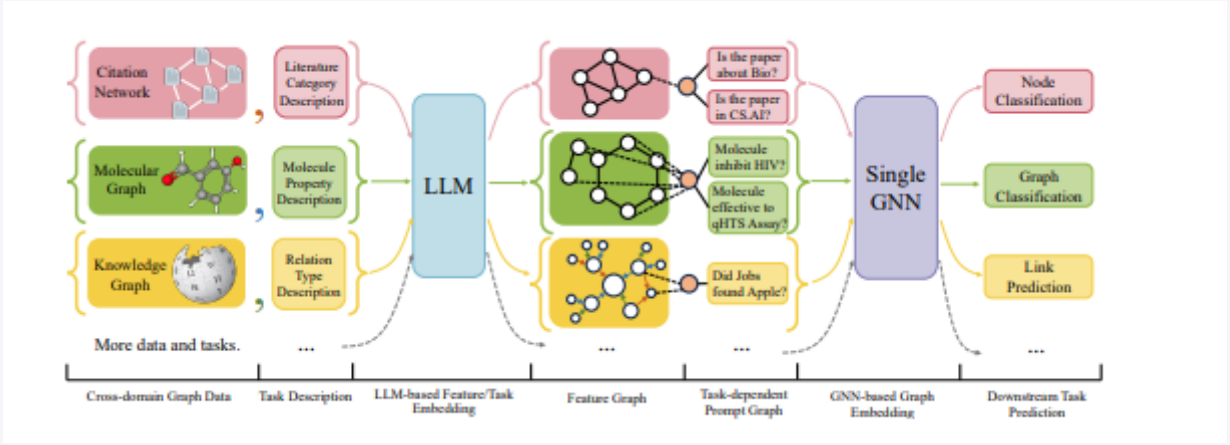
- A NOI prompt node hold the task description: (”Predict paper category”)
- Now add class nodes for example in below picture We have two classes Positive (P+ Node) and Negative (P- Node). For our Paper category we can have (cs.AI: Artificial Intelligence) Node and (cs.RO: Robotics) Node.
- These are connected to turning every task into a link prediction problem between prompt and class nodes.This enables in-context learning which means new tasks or classes can be added at inference by just describing them in text.So no fine tuning needed.
- The resulting ”Prompting Graph” is then fed to GNN and predictions are made via binary classification on class node embeddings.
- Formally for each class i let h_{c_i} be the final embedding of its class node after GNN processing.The models predicts the probability that the NOI belong to class i using a simple binary classifier:

  $$P[\text{NOI belongs to class } i] = \sigma(\text{MLP}(h_{c_i}))$$

  This design allows each class to be scored independently which is what makes zero shot prediction possible.



So the overall pipeline looks something like this



Main Results:
OFA is trained across nine diverse datasets which include citation, molecular, web and knowlege graphs. Key finding from that are

- **Supervised Learning:** OFA matches or exceeds specialized GNN (i.e GCN,GAT) using a simple model for all tasks.By looking at the table we can see that joint training even improves per dataset training suggesting cross domain knowledege transfer.

Table 2: Results on supervised learning (first).

Task type Metric	Cora Link AUC ↑	Cora[1] Node Acc ↑	PubMed Link AUC ↑	PubMed[1] Node Acc ↑	ogbn-arxiv[1] Node Acc ↑	Wiki-CS Node Acc ↑	HIV Graph AUC ↑
GCN	90.40±0.20	78.86±1.48	91.10±0.50	74.49±0.99	74.09±0.17	79.07±0.10	75.49±1.63
GAT	93.70±0.10	**82.76±0.79**	91.20±0.10	75.24±0.44	74.07±0.10	**79.63±0.10**	74.45±1.53
OFA-ind-st	91.87±1.03	75.61±0.87	98.50±0.06	73.87±0.88	75.79±0.11	77.72±0.65	73.42±1.14
OFA-st	94.04±0.49	75.90±1.26	98.21±0.02	75.54±0.05	75.54±0.11	78.34±0.35	78.02±0.17
OFA-e5	92.83±0.38	72.20±3.24	98.45±0.05	77.91±1.44	75.88±0.17	73.02±1.06	**78.29±1.48**
OFA-llama2-7b	94.22±0.48	73.21±0.73	**98.69±0.10**	77.80±2.60	77.48±0.17	77.75±0.74	74.45±3.55
OFA-llama2-13b	**94.53±0.51**	74.76±1.22	98.59±0.10	**78.25±0.71**	**77.51±0.17**	77.65±0.22	76.71±1.19

Table 3: Results on supervised learning (second).

Task type Metric	WN18RR Link Acc ↑	FB15K237 Link Acc ↑	PCBA Graph APR ↑
GCN	67.40±2.40	74.20±1.10	20.20±0.24
GIN	57.30±3.40	70.70±1.80	22.66±0.28
OFA-ind-st	97.22±0.18	**95.77±0.01**	22.73±0.32
OFA-st	96.91±0.11	95.54±0.06	24.83±0.10
OFA-e5	97.84±0.35	95.27±0.28	**25.19±0.33**
OFA-llama2-7b	98.08±0.16	95.56±0.05	21.35±0.94
OFA-llama2-13b	**98.14±0.25**	95.69±0.07	21.54±1.25

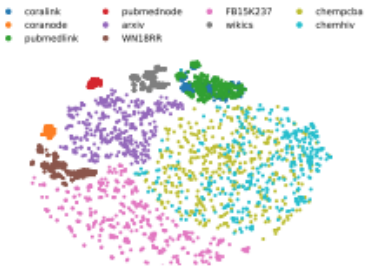


Figure 3: Output embedding space of NOI prompt nodes on all datasets for OFA-joint-st.

- **Few Shot or Zero Shot Learning:** It is quite astonshing to see that OFA acheives zero shot accuracy of (e.g 56.9% on Cora node classification with zero labeled examples) which is impossible for most baseline models.

Table 4: Few-shot and Zero-shot results (Acc) on ogbn-arxiv and Cora (Node-level).

# Way	ogbn-arxiv-5-way (Transductive)				Cora-2-way (Transfer)		
Task	5-shot	3-shot	1-shot	0-shot	5-shot	1-shot	0-shot
GPN	50.53±3.07	48.32±3.80	38.58±1.61	-	63.83±2.86	56.09±2.08	-
TENT	60.83±7.45	56.03±8.90	45.62±10.70	-	58.97±2.40	54.33±2.10	-
GLITTER	56.00±4.40	57.44±4.90	47.12±2.73	-	-	-	-
TLP-BGRL	50.13±8.78	46.21±7.92	35.81±8.58	-	81.31±1.89	59.16±2.48	-
TLP-SURGL	77.89±6.46	74.19±7.55	61.75±10.07	-	92.49±1.02	81.52±2.09	-
Prodigy	61.09±5.85	58.64±5.84	48.23±6.18	-	-	-	-
OFA-joint-lr	61.45±2.56	59.78±2.51	50.20±4.27	46.19±3.83	76.10±4.41	67.44±4.47	56.92±3.09

- **Cross task Generalization:** A single OFA model handles node, link and graph tasks simultaneously, outperforming task-specific methods like TLP-SURGL.
- **LLM Flexibility:** Perfomance scales with LLM size (OFA-llama2-13b $>$ OFA-st) but even smaller LLMs work well showing the framework isn't LLM dependent

OFA-ind-st	91.87±1.03	75.61±0.87	98.50±0.06	73.87±0.88	75.79±0.11	77.72±0.65	73.42±1.14
OFA-st	94.04±0.49	75.90±1.26	98.21±0.02	75.54±0.05	75.54±0.11	78.34±0.35	78.02±0.17
OFA-e5	92.83±0.38	72.20±3.24	98.45±0.05	77.91±1.44	75.88±0.17	73.02±1.06	**78.29±1.48**
OFA-llama2-7b	94.22±0.48	73.21±0.73	**98.69±0.10**	77.80±2.60	77.48±0.17	77.75±0.74	74.45±3.55
OFA-llama2-13b	**94.53±0.51**	74.76±1.22	98.59±0.10	**78.25±0.71**	**77.51±0.17**	77.65±0.22	76.71±1.19

Contributions:

- First unified framework for multi-task,cross domain graph learning
- Novel graph prompting mechanism enabling in-context learning on graphs.
- This also demonstrated the usefulness of natural language as it can unify heterogeneous graph features without losing structural information
- Experiments demonstrate that the approach remains effective in different scenarios such as supervised, few-shot or zero-shot.
- The ability to perform zero-shot learning on graphs could revolutionize applications where labeled data is less.

Reflection 2

Interesting ideas:
The idea of prompting a graph by literally adding nodes is brilliant its like giving the GNN **a query with few examples** as part of input graph. Also using LLMs not as predictors but as universal feature predictors is smart way to leverage their semantic power while preserving graph topology via GNNs.
Difficult concepts:
Some of the difficult concepts to understand are:

- How the model separates different domains in the embedding space while maintaining a single architecture.
- Understanding how the NOI prompt node and class nodes interact during message passing
- The mechanism by which few shot example are incorporated into graph structure.

Connections:

The work build connections upon

- Bridging the gap b/w foundation models and graph representation learning.
- Relates to prompt engineering in NLP domain but adapts it specifically for sturctural data.
- This paper acts more like a BERT moment in Graph domain.As Bert unified NLP via masked LM + tasked heads OFA unifies graph tasks via TAGs + prompt nodes.

Personal Takeaways:

- using text to unify different data modalities is a powerful paradigm that could extend beyond borders.
- The separation of representation via LLM and reasoning via GNN components creates a flexible architecture.
- In-context learning capabilities could significantly reduce the need for task-specific model training or fine-tuning.

Critical Thoughts:
My thoughts on this is that :

- Performance still lags for such architecture in comparison to methods specialized for specific tasks.
- The approach requires converting all features to text format, which is insufficient for large graphs.
- The method does not yet handle regression tasks, limiting its capability.

Future Work:

- Work can be extended to include regression tasks.
- Incorporate unsupervised pre-training techniques like contrastive learning.
- Scale to larger datasets approaching the scale of LLM training data.

Paper 3: Title

Paper URL: https://openreview.net/forum?id=K9xHDD6mic&noteId=gw1Nl5pq4H

Paper Title: Graph Mixture of Experts: Learning on Large-Scale Graphs with Explicit Diversity Modeling

Authors: Haotao Wang, Ziyu Jiang, Yuning You , Yan Han , Gaowen Liu3 , Jayanth Srinivasa Ramana Rao Kompella , Zhangyang Wang

Conference: NeurIPS 2023

Goal / Problem:
The paper addresses the fundamental challenge in Graph Neural Networks Learning **"How to effectively scale GNN model's capacity to leverage diverse graph structures without**

compromising inference efficiency". Real world graphs often contain heterogeneous nodes and edges with varying structural patterns.Standard GNN (e.g GCN) force all nodes to share the same aggregation mechanism regardless of neighborhood differences, limiting their ability.

Proposed Method:
The authors of the paper proposed Graph Mixture of Experts(GMOE) which adapts the Mixture of Experts idea to the graph neural network domain.
Instead of having a single aggregation function per GNN layer each GMOE layer contains multiple GNN experts where

- Each expert is a message passing GNN such as GCN or Graph Sage
- Experts differ in aggregation hop size (e.g hop1 and hop2 neighbors) Each node dynamically selects a small subset of experts using a gating network

The gating network operates nodes wise meaning each node chooses which experts to use based on its own features.Also the model use sparse routing based on top k experts so only a few experts are activated per node, keeping computation efficient.

$$G(h_i) = \text{Softmax}(\text{TopK}(W_g h_i + \epsilon \cdot \text{SoftPlus}(W_n h_i)))$$

One important thing is that to prevent the model from collapsing onto single expert the authors add

- **Importance Loss:** Encourages balanced expert usage

$$\text{Importance}(H) = \sum_{h_i \in H, g \in G(h_i)} g, L_{\text{importance}}(H) = CV(\text{Importance(H)})^2$$

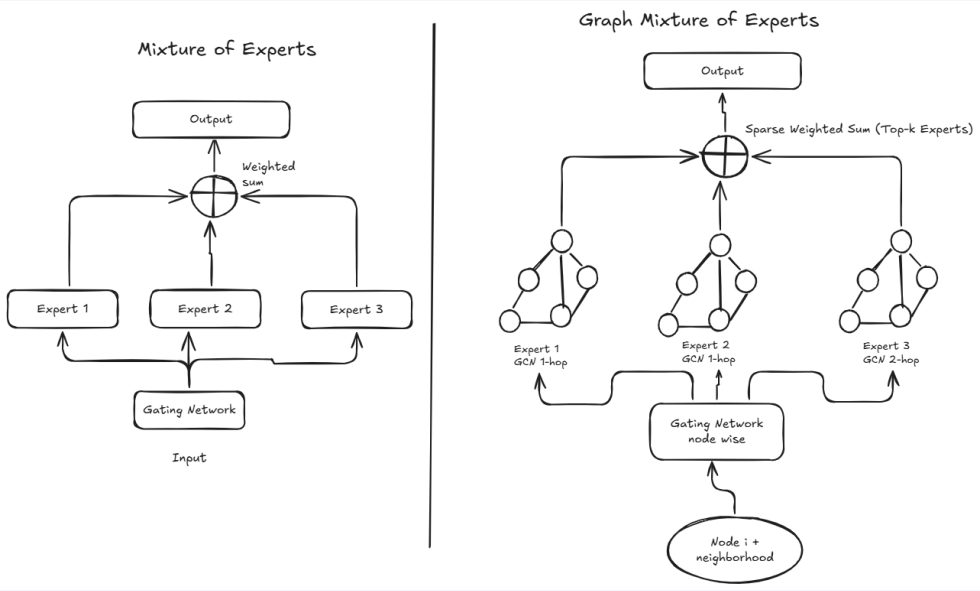
- **Load Balancing Loss:** Ensures experts receive similar number of nodes

$$L_{load}(H) = CV\left(\sum_{h_i \in H, p \in P(h_i, o)} p\right)^2$$

The final loss combines both

$$L = L_{EM} + \lambda(L_{load}(H) + L_{importance}(H))$$

where L_{EM} denotes task-specific MOE expectation maximizing loss.



GMOE can be plugged into standard backbones like GCN and GIN, resulting in models such as GMoE-GCN and GMoE-GIN.

Main Results:

The Graph Mixture of Experts model was tested on 10 different datasets from OGB benchmark covering node classification, link prediction, graph classification, graph regression.

The key findings for Graph Classification are that GMoE-GCN outperforms baseline GCN across all molecular prediction datasets.

The gains are quite significant because GMoE ability to dynamically select appropriate aggreation experts based on local neighborhood structure proved valuable for these complex molecular graphs.The model could adaptively choose between 1-hop or 2-hop experts depending on node connectivity patterns.

Model	ogbg-molbbbp	ogbg-molhiv	ogbg-moltoxcast	ogbg-moltox21
	ROC-AUC (↑)			
GCN	68.87 ± 1.51	76.06 ± 0.97	63.54 ± 0.42	75.29 ± 0.69
GMoE-GCN	**70.28** ± 1.36 (+1.41)	**77.87** ± 1.03 (+1.81)	**64.12** ± 0.61 (+0.58)	**75.45** ± 0.58 (+0.16)

Regression Task: This is another area where GMoE also improved performance as can be seen in below table.The RMSE value improved for both ogbg-molesol and ogbg-molfreesol.

According to authors the regression tasks benefit particularly from GMoE ability to model structural diversity for these complex datasets. The study revealed that smaller molecular graphs like ogbg-molfreesolv primarily benefit from 1-hop experts while larger molecules require 2-hop experts for optimal performance.

Model	ogbg-molesol	ogbg-molfreesolv
	RMSE (↓)	
GCN	1.114 ± 0.036	2.640 ± 0.239
GMoE-GCN	**1.087** ± 0.043 (-0.027)	**2.500** ± 0.193 (-0.140)

Enhanced Performance with Pretraining: The paper also proved that when combined with Graph MAE pretraining, GMoE-GIN demonstrated even stronger results as shown in below table. It outperformed standard GIN as the ROC-AUC score jumped from 65.5 to 66.93 for ogbg-molbbbp dataset.

This significant jump can be result of good structural representations that benefit from GMoE specialized aggregation strategies.

Model	ogbg-molbbbp	ogbg-molhiv	ogbg-moltoxcast	ogbg-moltox21	Average
			ROC-AUC (↑)		
GIN	65.5 ± 1.8	75.4 ± 1.5	63.3 ± 1.5	74.3 ± 0.5	69.63
GMoE-GIN	**66.93** ± 1.72 (+1.43)	**76.14** ± 1.03 (+0.74)	62.86 ± 0.37 (-0.44)	**74.76** ± 0.66 (+0.46)	**70.17** (+0.54)
GIN+Pretrain	**69.94** ± 0.92	76.1 ± 0.8	62.96 ± 0.55	73.85 ± 0.64	70.71
GMoE-GIN+Pretrain	68.62 ± 1.02 (-1.32)	**76.9** ± 0.9 (+0.8)	**64.48** ± 0.50 (+1.18)	**75.25** ± 0.78 (+1.40)	**71.31** (+0.6)

Node Prediction: The GMoE also show good improvements in node prediction task. Although the authors didnt provide exact reason but it could be due to the GMoE's ability to intelligently select aggregation experts tailored to each node as discussed in Section 3.1 in paper.

Model	ogbn-protein ROC-AUC(↑)	ogbn-arxiv Acc(↑)
GCN	73.53 ± 0.56	71.74 ± 0.29
GMoE-GCN	**74.48** ± 0.58 (+0.95)	**71.88** ± 0.32 (+0.14)

Link Prediction Tasks:
The paper also proves that GMOE-GCN models have also improved the link prediction tasks.The below table verifies that as 0.89% increase in the Hits@20 scores for ogbl-ddi dataset.

Model	ogbl-ppa Hits@100(↑)	ogbl-ddi Hits@20(↑)
GCN	18.67 ± 1.32	37.07 ± 0.051
GMoE-GCN	**19.25** ± 1.67 (+0.58)	**37.96** ± 0.082 (+0.89)

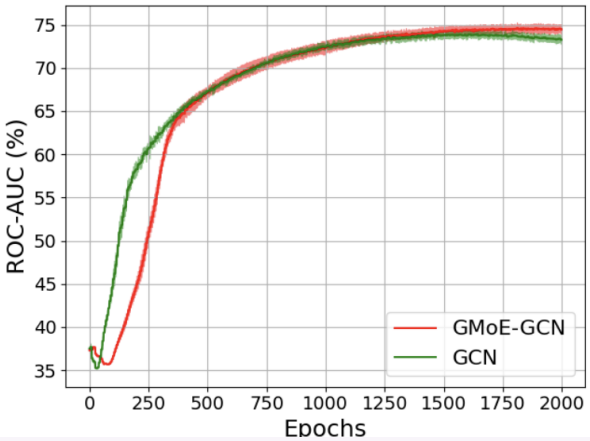
Ablation Studies
Structural Diversity: The ablation study on ogbg-molhiv (larger graph) and ogbg-molfreesolv (smaller graph) confirmed that large graphs perform better with 2-hop experts while smaller graph performs better with 1-hop expert.

Dataset	Metric	Average #Nodes	$n=m=4$ $k=4$	$n=m=4$ $k=1$	$n=m=8$ $k=4$	$n=8, m=0$ $k=4$
ogbg-molfreesolv	RMSE (↓)	8.7	2.856 ± 0.118	**2.500** ± 0.193	2.532 ± 0.231	2.873 ± 0.251
ogbg-molhiv	ROC-AUC (↑)	25.5	76.67 ± 1.25	76.41 ± 1.72	77.53 ± 1.42	**77.87** ± 1.03

Sparse Expert Selection:
As depicted in below table results using fewer experts than available (k¡n) improved both efficiency and generalization.This suggests that expert specialization where each expert focuses on a specific type of neighborhood pattern leads to better performance than having all experts process all nodes.

Dataset	Metric	Average #Nodes	$n=m=4$ $k=4$	$n=m=4$ $k=1$	$n=m=8$ $k=4$	$n=8, m=0$ $k=4$
ogbg-molfreesolv	RMSE (↓)	8.7	2.856 ± 0.118	**2.500** ± 0.193	2.532 ± 0.231	2.873 ± 0.251
ogbg-molhiv	ROC-AUC (↑)	25.5	76.67 ± 1.25	76.41 ± 1.72	77.53 ± 1.42	**77.87** ± 1.03

GMoE vs Epoch: In the below figure although GMoE-GCN converges more slowly that the single expert GCN requiring more training epochs, it ultimately achieves superior performance. This happens because each expert receives fewer updates initially thus are weaker,but after sufficient training, each expert becomes highly specialized for its assigned structural pattern



Load Balancing:
From the table below it is clear that for load balancing loss co-efficient λ showed the proper balance at $(\lambda = 0.1)$ thus improving performance by 2.58% ROC-AUC compared to no load balancing at $(\lambda = 0)$ demonstrating that preventing **"GMoE collapse"** is essential for model effectiveness.

λ	0	0.1	1
ROC-AUC	72.87±1.04	**75.45**±0.58	75.27±0.33

Improved State-of-the-art-models:
In Seection 4.3 (Observation 5) GMoE was integrated with Neural FingerPrints (top 5 performer ogbg-molhiv leaderboard)

Leaderboard for ogbg-molhiv

The ROC-AUC score on the test and validation sets. The higher, the better.

Package: >=1.1.1

Rank	Method	Ext. data	Test ROC-AUC	Validation ROC-AUC	Contact	References	#Params	Hardware	Date
1	**HyperFusion**	No	0.8475 ± 0.0003	0.8275 ± 0.0008	Xinwei Zhang(Tsinghua University)	Paper, Code	5,908,027	RTX 3080	Feb 24, 2024
2	**PAS+FPs**	No	0.8420 ± 0.0015	0.8238 ± 0.0028	Xu Wang(4Paradigm)	Paper, Code	26,706,953	RTX3090	Feb 22, 2022
3	**HIG**	No	0.8403 ± 0.0021	0.8176 ± 0.0034	Yan Wang (Tencent Youtu Lab)	Paper, Code	1,019,408	Tesla V100 (32GB)	Dec 28, 2021
4	**DeepAUC**	No	0.8352 ± 0.0054	0.8238 ± 0.0061	Zhuoning Yuan (UIowa)	Paper, Code	3,444,509	Tesla V100 (32GB)	Oct 10, 2021
5	FingerPrint+GMAN	No	0.8244 ± 0.0033	0.8329 ± 0.0039	Jiaxin Gu	Paper, Code	1,444,110	Tesla V100 (32GB)	Jul 8, 2021
6	**Neural FingerPrints**	No	0.8232 ± 0.0047	0.8331 ± 0.0054	Shanzhuo Zhang (PaddleHelix & PGL)	Paper, Code	2,425,102	Tesla V100 (32GB)	Mar 15, 2021

The combination achieved 82.72% + 0.53% accuracy while maintaining same computational resources outperforming the original model by 0.4% demonstrating GMoE ability to enhance even sophisticated existing architectures.
Contributions:
This paper has contributed in many ways to growing field of GNN research.
It is first application of sparse Mixture of Experts to scale Graph Neural Networks in an end to end fashion.
The experts in this framework capture both structural diversity (1-hop and 2-hop) and data heterogeneity (molecules vs protiens).
The paper has shown consistent gain across node,link and graph tasks and pretrained/non-pretrained settings.
The research tries to match baseline computation resources while improving accuracy.

Reflection 3

Interesting ideas:
The **hop-2 expert** is a clever way to inject long range info without deeper GNNs.
Also the **Sparse Expert Selection** using fewer experts than available is a fantastic that improves both efficiency and generalization is counter intuitive but valuable shows that a specialized experts for specific task are more useful than all experts process all nodes.Its like a same concept of using **Dropout** (using lesser neurons) that are given during training to reduce overfitting and

reduce generalization.
Difficult concepts:
In order to understand gating function with noise injection you need to have critical understand of how Mixture of Experts (MOE) works. The load balancing concept mathematically is difficult to grasp first although its just a regularizer to keep our right experts employed. Also why the hop-2 experts don't explode in FLOPs the MixHop concept was difficult to grasp.
Connections:
GMoE actually took the concept of sparse activation from MOE but tailored gating to graph topology.It relates to heterogeneous GNN but took a more dynamic approach by routing nodes to experts rather them having fixed heterogeneous processing paths.
Personal Takeaways:
GMoE per node adaptivity is next level step after the Graph Attention Networks(GAT) that use attention mechanism. This paper showcases how combining ideas between different domain can yield significant advances.The balance between model capacity vs inference efficiency is especially useful for industrial applications like recommendation systems and molecular virtual screening where throughput matters.
Also gating uses node features and misses neighbour info it could miss structural cues.
Critical Thoughts:
Although the results are strong the paper primarily elevates on fundamental backbones (GCN,GIN).It is unclear for now how GMoE will perform when applied to more sophisticated architectures.Also the smaller convergence might be practical limitation for some applications
Future Work:

- This work could be further extended to combining GMoE with attention based GNNs i.e GAT,Graph Transformer which could be primarily impactfull.
- Also exploring semi-supervised learning scenarios and investigating domain-specific expert routing mechanisms would strengthen practical applicability.
- Also making the model to choose how many experts are needed for specific task (dynamic k per node) would be area to explore.

Paper 4: Title

Paper URL: https://openreview.net/forum?id=haVPmN8UGi

Paper Title: GraphVis: Boosting LLMs with Visual Knowledge Graph Integration

Authors: Yihe Deng, Chenchen Ye, Zijie Huang, Mingyu Derek Ma, Yiwen Kou, Wei Wang

Conference: NeurIPS 2024

Goal / Problem:
This paper addresses the fundamental challenge in LLM development that **"How can we effectively integrate structured knowledge from Knowledge Graphs while preserving its inherent structure"**.Current KG enhanced LLM approach either linearizes KG triplets into text (losing structural information) or uses GNN embeddings (with limited ability). Additionaly this paper explores how this cross-modal integration can simultaneously enhance LVLM's capabilities on visual question answering tasks, particularly for images that contains graph like structure(i.e diagrams,charts etc).
Proposed Method:
The authors introduced a concept of GraphVi's that:

- Instead of linearizing KGs into text, GraphVi's converts relevant subgraphs into images

using Graphviz preserving structural

-

Main Results:

Contributions:

Reflection 4

Interesting ideas:

Difficult concepts:

Connections:

Personal Takeaways:

Critical Thoughts:

Future Work:

Paper 5: Title

Paper URL: https://openreview.net/forum?id=ayF8bEKYQy
Paper Title: _____
Authors: _____
Conference: _____ **Year:** _____
Goal / Problem:

Proposed Method:

Main Results:

Contributions:

Reflection 5

Interesting ideas:

Difficult concepts:

Connections:

Personal Takeaways:

Critical Thoughts:

Future Work:

Section 2: Quiz Templates

In this second task, you will watch the first full 4 lectures of the Playlist of the "Deep Graph Learning Course" from **1.1** to **4.6**. You can also complete the first two tutorials available at https://github.com/basiralab/dgl.

For each lecture, you are required to design **ten multiple-choice questions**. Questions can include **figures and equations**.

- **Five questions** should be based directly on the lecture material.
- The remaining **five questions** should be more challenging, assessing the learner's ability to *apply or extend* the lecture content and tools to related or novel problems.

Questions should be balanced in difficulty so that a well-prepared student can **comfortably**

answer approximately 60% of them.
Below are example quiz question templates you can duplicate or modify. Each question supports: - Multiple-choice options (A–D) - Optional equation (‘LaTeX‘) or image - Marking of the correct answer using ‘`\correct{...}`‘.
—

Quiz Question: 1

What can you not skip to define a graph?

$$G = (A, V, E)$$

where A = Adjacency Matrix V = Node embeddings matrix E = Edges embeddings matrix

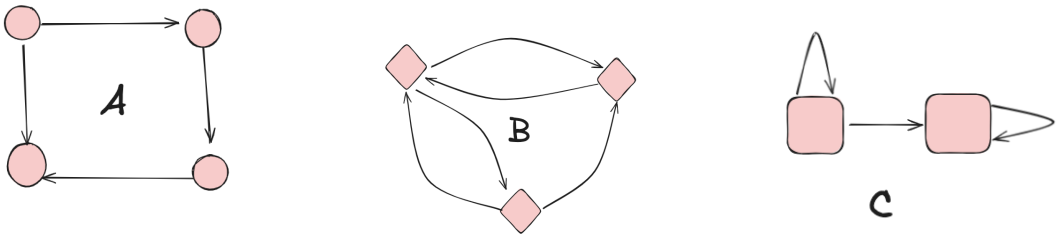
A. Node Embeddings Matrix V

B. Edges Embeddings Matrix E

C. **Adjacency Matrix** A

Quiz Question: 2

Which one is a simple graph?



A. **A**

B. B

C. C

Quiz Question: 3

In a sparse graph _____ percent of edges are zero?

A. 60%

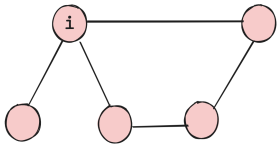
B. 70%

C. 80%

D. **90%**

Quiz Question: 4

What is degree of node **"i"** in the image?



A. 1

B. 2

C. **3**

D. 4

Quiz Question: 5

According to the slide, what is a major limitation of 'shallow' embedding approaches regarding node data?

A. **They cannot utilize node, edge, and graph features.**

B. They are too computationally expensive for small graphs.

C. They cannot represent graph topology.

D. They cannot process directed edges.

Quiz Question: 6

You are designing a recommendation system for a social network where thousands of new users join every day.Why would a GNN be preferred over a shallow embedding like Matrix Factorization?

A. GNNs are faster to train on static datasets.

B. GNNs remove the need for an adjacency matrix.

C. Shallow embeddings cannot handle user IDs.

D. **GNNs support inductive learning allowing them to generate embeddings for new users without retraining.**

Quiz Question: 7

In a simple GCN why do we add Identity matrix I to Adjancency Matrix A

$$\hat{A} = A + I$$

A. Due to other connectivity with other nodes so the it can update other nodes embeddings.

B. **Due to self connectivity of a particular node so it can update its own embedding**

C. It prevents the original node features from being completely replaced by aggregated neighbor features.

D. It ensures numerical stability during message passing and improves the propagation of gradients.

Quiz Question: 8

The node embedding update rule is represented by

$$h_{k+1} = a(\beta_k +_k h_k^{(n)} +_k agg[n,k])$$

what does a stands for:

A. **Activation Function**

B. Aggregation function

C. Adjacency Matrix

D. Input Embedding

Quiz Question: 9

During permutation when the graph change the order of nodes the graph topology what does a stands for:

A. reverses

B. **remains same**

C. normalizes the node features

D. applies a non-linear transformation

Quiz Question: 10

In Graph Neural Networks, message passing updates node embeddings using which operation?

A. Backpropagation through dense layers

B. **Aggregation of neighbour features**

C. Random feature dropout

D. Batch normalization

Quiz Question: 11

What is a key advantage of using bias nodes instead of self-loops or node features to represent bias parameters in the parameter graph?

A. It reduces the total number of edges in the graph

B. It reduces memory requirements by 50% compared to other representations

C. **It allows bias parameters within the same layer to communicate with each other in a single GNN layer**

D. It makes the graph acyclic, which is required for theoretical guarantees

Taks 3 (BONUS): Open GNN Mini-Competition Design & Launch on GitHub

This section provides a step-by-step guide to creating a GNN mini-challenge on GitHub that is simple to join but difficult to solve, similar to a mini-Kaggle competition.
Important: Those who implement Task 3 will join a high-level research project to submit to a Class A1 conference on AI: NeurIPS 2026 (deadline in May).

1 Define Your Challenge Covering DGL 1.1 to 4.6

- **Problem type:** Node classification, graph classification, graph generation, link prediction, etc. The problem you choose should be one that was covered in DGL lectures 1.1 through 4.6 (e.g., *sampling methods, message passing methods, etc*).

  You are encouraged to be creative and define an engaging problem to solve, based on the five research papers you reviewed above or any other GNN papers you find interesting. The problem should be solvable using the methods covered in DGL lectures 1.1–4.6.

- **Dataset:** Small enough to download easily, but complex enough to be challenging.

- **Objective metric:** Choose a metric that is difficult to optimize (e.g., F1-score for imbalanced classes, mean average precision, or log-loss for probabilistic models).

- **Constraints:** Consider rules like “no external data,” “must run under X minutes,” or “use only linear models” to increase difficulty without complicating the setup.

2 Prepare the Dataset

- Split the dataset into:
  - `train.csv` — includes features and labels.
  - `test.csv` — features only (participants predict labels here).
- Optionally keep a hidden validation set for final scoring.
- Add complexity:
  - Noise, missing values, unbalanced classes.
  - Subtle feature interactions.
  - Large feature space relative to dataset size.
  - Other 🤓

3 Repository Structure

A recommended GitHub repository structure:

```
gnn-challenge/
 data/
    train.csv
    test.csv
 submissions/
    sample_submission.csv
 starter_code/
    baseline.py
    requirements.txt
 README.md
 scoring_script.py
 LICENSE
```

4 Starter Code: Baseline Model

Provide a simple baseline model in `baseline.py` to get participants started:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score

# Load data
train = pd.read_csv('../data/train.csv')
X = train.drop('target', axis=1)
y = train['target']

# Split into train/validation
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state
    =42)

# Train a baseline model
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
```

```

# Evaluate
y_pred = clf.predict(X_val)
score = f1_score(y_val, y_pred, average='macro')
print(f'Validation F1 Score: {score:.4f}')

# Make predictions on test set
test = pd.read_csv('../data/test.csv')
test_preds = clf.predict(test)
pd.DataFrame({'id': test['id'], 'target': test_preds}).to_csv('../submissions/
    sample_submission.csv', index=False)
```

Listing 1: starter_code/baseline.py

5 Scoring Script

A scoring script for organizers (`scoring_script.py`):

```
import pandas as pd
from sklearn.metrics import f1_score
import sys

# Usage: python scoring_script.py submissions/file.csv

submission_file = sys.argv[1]

# Load submission
submission = pd.read_csv(submission_file)

# Load ground truth (hidden)
truth = pd.read_csv('data/test_labels.csv')

# Compute F1 score
score = f1_score(truth['target'], submission['target'], average='macro')
print(f'Submission F1 Score: {score:.4f}')
```

Listing 2: scoring_script.py

6 Hosting Submissions and Leaderboard

Options on GitHub:

1. **Manual submission:** Participants submit CSV files via pull requests. Maintain a leaderboard manually in `leaderboard.md`.
2. **Automated scoring using GitHub Actions:** Example workflow:

```
name: Score Submission
on:
  pull_request:
    paths:
      - 'submissions/*.csv'
jobs:
  score:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
```

```
          python-version: '3.10'
      - name: Install dependencies
        run: pip install -r starter_code/requirements.txt
      - name: Run scoring
        run: python scoring_script.py submissions/$PR_SUBMISSION
      - name: Update leaderboard
        run: python update_leaderboard.py
```

Listing 3: GitHub Actions workflow snippet

7 Tips for a Simple but Difficult Challenge

- Dataset should look easy, but the metric or feature interactions make it tricky.
- Avoid huge files—small, clever datasets are enough.
- Provide starter code to reduce the entry barrier.
- Automate leaderboard updates for instant feedback.